

Bitte beantworten Sie die Fragen in den dafür vorgesehenen Feldern. Wenn Sie keinen Platz mehr für eine Antwort haben, fahren Sie bitte auf der Rückseite fort und geben an wo es weiter geht.
Abgesehen von Internetfähigen Geräten sind alle Hilfsmittel zugelassen.
Schreiben Sie Ihre Klausur bitte nicht mit Bleistift und lassen Sie die Klausur bitte geheftet.
Sie haben 90 Minuten Zeit. Mit 43 Punkten gilt die Klausur als bestanden.

Vor- und Nachname (in Druckbuchstaben): _____

Matrikelnummer und Studiengang: _____

Aufgabe Nr.:	Punktzahl:	Davon erreicht:
1	6	
2	4	
3	14	
4	8	
5	8	
6	8	
7	7	
8	8	
9	7	
10	9	
11	11	
Summe	90	

Note und Unterschrift des Erstprüfers: _____

Unterschrift des Zweitprüfers: _____

Lernraum I

1. (6 Punkte) Geben Sie die Laufzeit für die folgende Funktion g in der Θ -Notation an. Es sollte ersichtlich sein wie Sie auf diese Laufzeit gekommen sind (bspw. sehr kurze Begründung).

```
def g(n):
    x = 0
    while n > 1:
        for i in range(0, 10):
            x += 1
        n = n // 3
    return x
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

2. (4 Punkte) Geben Sie für jede der folgenden vier Funktionen die vereinfachte O-Notation in der zweiten Spalte der Tabelle an und beantworten Sie dann die entsprechende “Wahr oder Falsch” Frage zu dieser Funktion durch Einkreisen von “W” bzw. “F”.

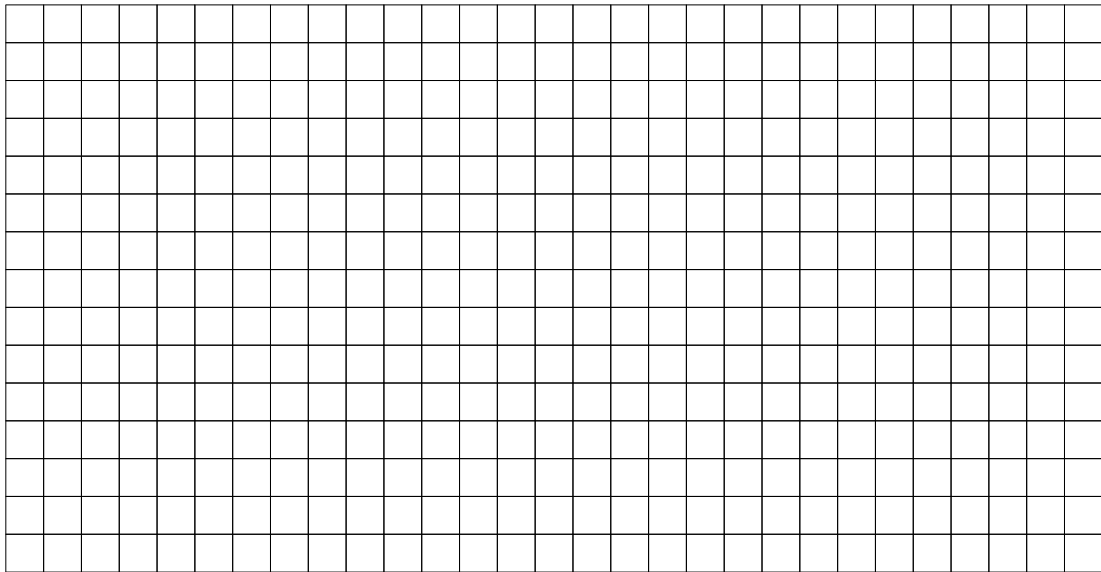
Funktion	Vereinfachte O-Notation	Wahr (W) oder Falsch (F)? (Einkreisen)
$a(n) = 42 \log_3(n)$		$a(n)$ ist in $\Theta(n)$ W oder F
$b(n) = 4n + 3000n^2$		$b(n)$ ist in $\Omega(n \log n)$ W oder F
$c(n) = 10C_1 + C_2, C_1, C_2 \in \mathbb{N}$		$c(n)$ ist in $O(n^2)$ W oder F
$d(n) = \frac{n}{4} + \log_2(2n)$		$d(n)$ ist in $\Omega(\log n)$ W oder F

Lernraum II

3. (14 Punkte) Stellen Sie sich vor, Sie möchten eine Min-Prioritätswarteschlange implementieren. Anstelle eines Heaps möchten Sie als abstrakten Datentyp allerdings ein aufsteigend sortiertes Array nutzen. Welche Laufzeiten werden die Operationen `add()`, `removeMin()` und `peekMin()` haben und wie können diese umgesetzt werden? Nehmen Sie dabei an, dass vor dem Aufruf der Operation das Array aufsteigend sortiert ist und bedenken Sie, dass am Ende der Operation das Array ebenfalls sortiert sein muss. Beschreiben Sie die Implementierungen der drei Operationen (als Pseudocode oder in ein paar Sätzen/Bulletpoints) und geben Sie deren Laufzeit in der O-Notation an. Gehen Sie bei `add()` davon aus, dass im Array noch genügend Platz für das einzufügende Element ist.

[illegible]

4. (8 Punkte) Geben Sie eine Einfügereihenfolge für die Schlüssel 1, 2, 3, 4, 5, 6, 7 an, sodass beim Einfügen in einen leeren AVL-Baum **keine Rotation notwendig** ist. Zeichnen Sie den resultierenden AVL-Baum nach dem vollständigen Einfügevorgang.



5. (8 Punkte) Sie sollen ein Ticketsystem für kostenloses WLAN für Passagiere am Flughafen entwickeln. Durch Eingabe der Nummer des Reisepasses soll ein eindeutiges Passwort für den Inhaber des Passes generiert, intern abgespeichert und ausgegeben werden. Welchen abstrakten Datentyp und welche Datenstruktur wählen Sie, um das Passwort für jeden Passagier abzuspeichern?

Zwei Passagiere mit den Passnummern:

1. CZ6311T47
2. DE5329S52

haben das Ticketsystem bereits genutzt. Für den ersten Passagier wurde das Passwort 3241 und für den zweiten Passagier das Passwort 5436 generiert. Geben Sie kurz an wie diese Daten in der Datenstruktur gespeichert werden (1-2 Sätze oder eine Zeichnung inklusive einem erklärenden Satz).

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

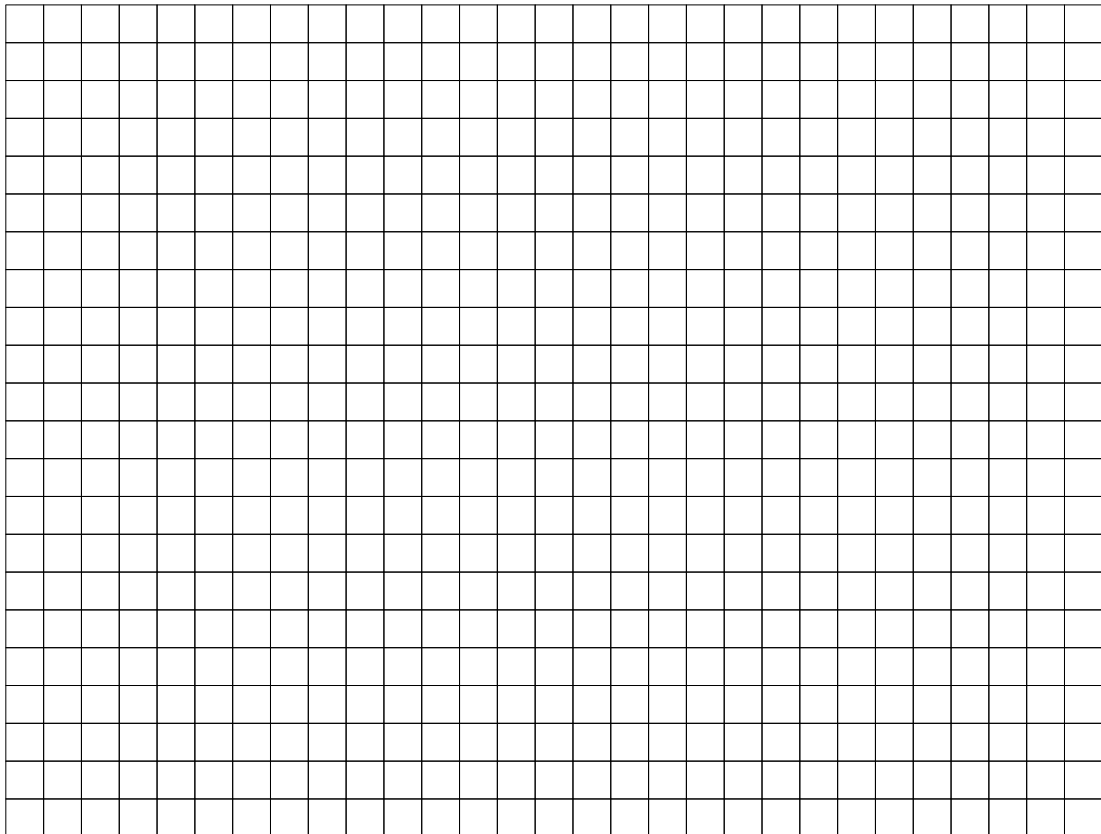
Lernraum III

6. (8 Punkte) Erweitern Sie den iterativen Algorithmus der Tiefensuche (DFS) so, dass er die Anzahl der vom Startknoten **start** aus erreichbaren Knoten zurückliefert (inklusive **start** selbst). Nutzen Sie dazu den untenstehenden Code.

Geben Sie an, an welchen Stellen im Code Änderungen oder Ergänzungen vorgenommen werden müssen. Es reicht, die betreffenden Zeilennummern zu nennen und den jeweils hinzugefügten oder geänderten Code in Pseudocode (oder einer Sprache Ihrer Wahl) anzugeben.

Algorithm 1 Iterative Tiefensuche (DFS)

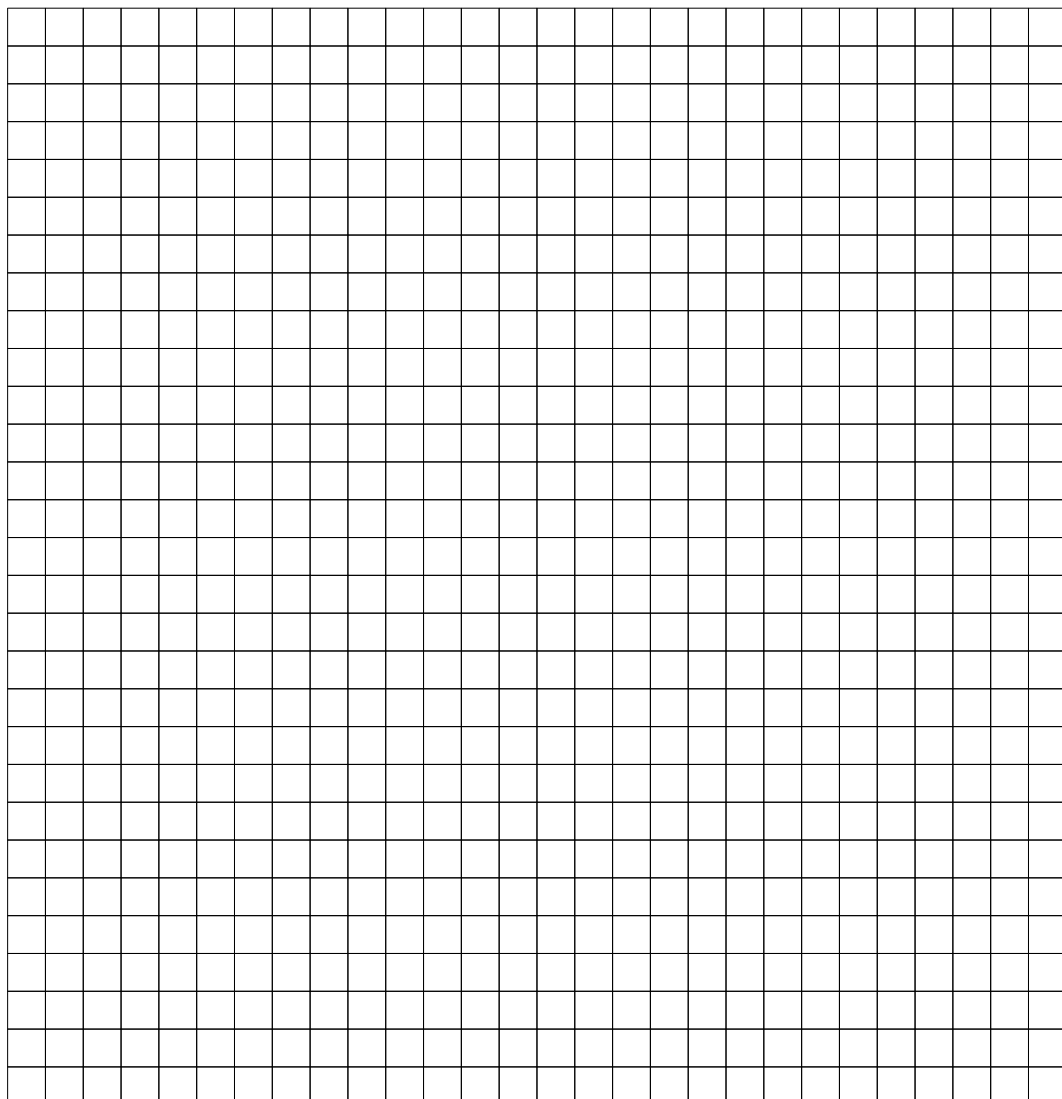
```
1: function dfs(Graph graph, Vertex start) {
2:   Stack<Vertex> perimeter = new Stack<>();
3:   Set<Vertex> visited = new Set<>();
4:
5:   perimeter.add(start);
6:
7:   while (!perimeter.isEmpty()) {
8:     Vertex from = perimeter.remove();
9:     if (!visited.contains(from)) {
10:      for (Edge edge : graph.edgesFrom(from)) {
11:        Vertex to = edge.to();
12:        if (!visited.contains(to))
13:          perimeter.add(to);
14:      }
15:      visited.add(from);
16:    }
17:  }
18: }
```



7. Modellieren Sie den Aufbau eines Badezimmer-Schranks als gerichteten Graphen und geben Sie die Minstdauer für den Gesamtvorgang an unter der Annahme, dass parallel durchführbare Schritte die Dauer der einzelnen Schritte nicht verlängern. Die Abhängigkeiten der Arbeitsschritte sind wie folgt:

1. Bauteile auspacken: Muss als erstes gemacht werden, keine Abhängigkeiten. Dauer: 3 Minuten
2. Schrauben sortieren: Unabhängig durchführbar. Dauer: 2 Minuten
3. Seitenteile montieren: Abhängig von "Bauteile auspacken". Dauer: 5 Minuten
4. Rückwand anschrauben: Abhängig von "Seitenteile montieren". Dauer: 4 Minuten
5. Böden einbauen: Abhängig von "Seitenteile montieren". Dauer: 3 Minuten
6. Türen montieren: Abhängig von "Böden einbauen". Dauer: 5 Minuten
7. Griffe anschrauben: Abhängig von "Türen montieren" und "Schrauben sortieren". Dauer: 2 Minuten
8. Schrank ausrichten: Abhängig von "Rückwand anschrauben". Dauer: 2 Minuten
9. Reinigung: Abhängig von "Griffe anschrauben" und "Schrank ausrichten". Dauer: 3 Minuten

(a) (6 Punkte) Graph:



(b) (1 Punkt) Gesamtdauer des Vorgangs:

.....

8. Ordnen Sie jeder der folgenden Anwendung einen Algorithmus zu. Wählen Sie aus den folgenden Algorithmen:

- Dijkstra's Algorithmus
- Breitensuche
- Erreichbarkeitsproblem
- Topologische Sortierung
- Insertionsort
- Prims Algorithmus
- Dynamische Programmierung
- Heapsort

(a) (2 Punkte) Bestimmung des kürzesten Pfads in ungewichteten Netzwerken, z. B. in sozialen Netzwerken.

.....

(b) (2 Punkte) Bestimmung der kostengünstigsten Verbindung zwischen Servern eines Computernetzwerks.

.....

(c) (2 Punkte) Bestimmung des kürzesten Wegs zwischen zwei Städten.

.....

(d) (2 Punkte) Berechnung von Abhängigkeitsreihenfolgen, z. B. bei der Planung von Bauprojekten.

.....

9. Weighted Quick-Union mit Pfadkomprimierung:

- (a) (6 Punkte) Rufen Sie `union(4, 15)` auf die folgenden beiden Bäume auf. Nutzen Sie dabei Weighted Quick-Union mit Pfadkomprimierung. Zeichnen Sie den sich ergebenden Baum. Hinweis: Es dürfte hilfreich sein, Zwischenschritte zu zeichnen.

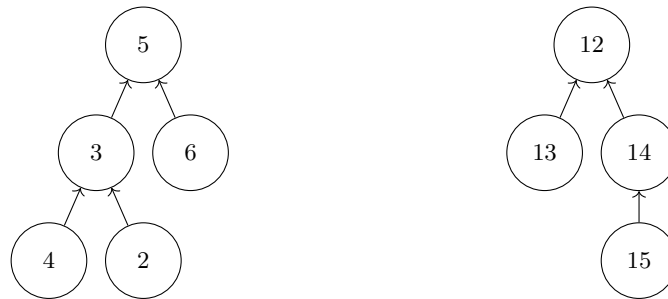
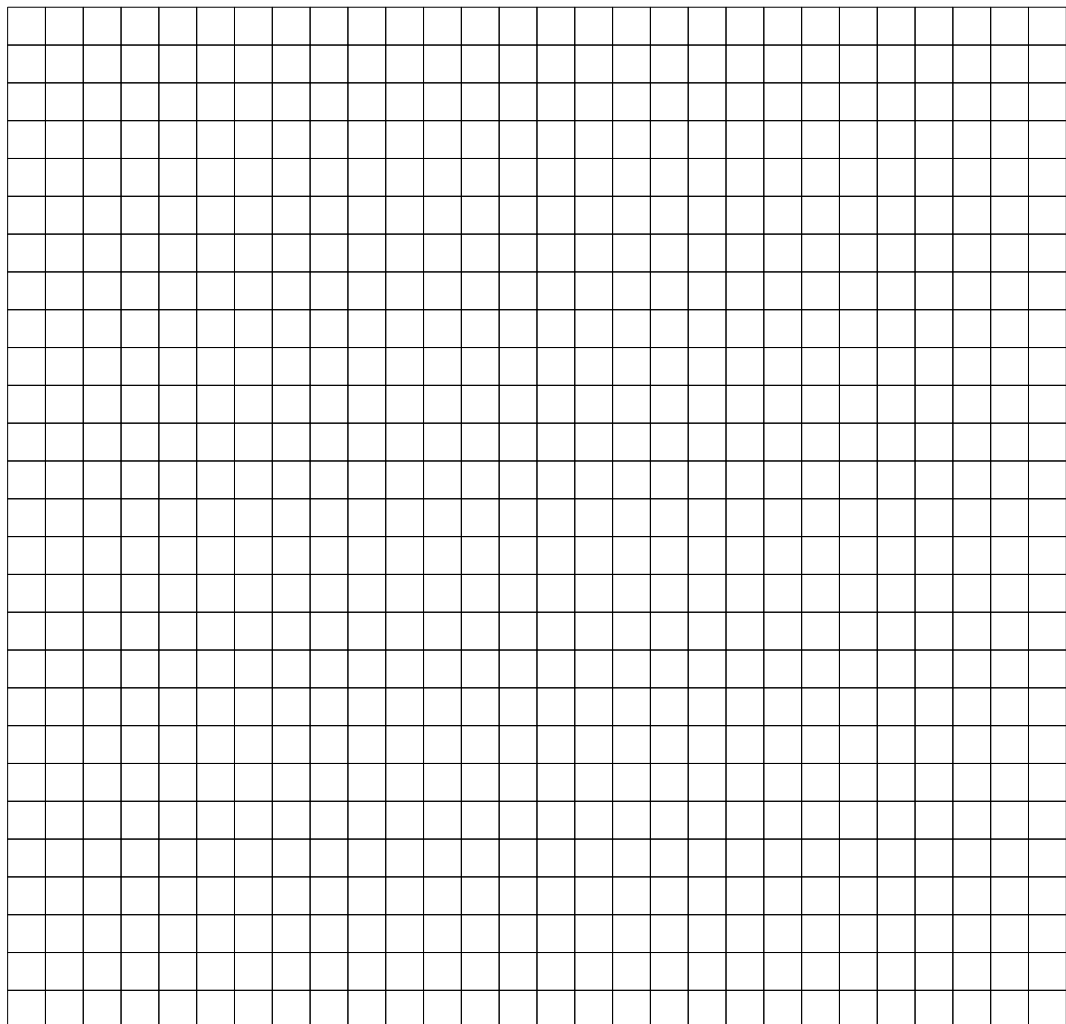


Abbildung 1: Darstellung von zwei disjunkten Mengen auf denen Sie `union(4, 15)` aufrufen sollen.



- (b) (1 Punkt) Welche Laufzeit hat die Operation `find(12)` auf dem neuen Baum?

.....

Lernraum IV

10. Wählen Sie für jedes der folgenden Szenarien den passenden Sortieralgorithmus aus und begründen Sie Ihre Wahl in einem kurzen Satz.

Zur Wahl stehen: Insertionsort, Selectionsort, Mergesort, Quicksort, Heapsort, Bucketsort

- (a) (3 Punkte) Eine weltweit bekannte Online-Videoplattform zeigt in einem Liveticker, wie viel Zuschauer gerade einen Livestream schauen. Alle paar Sekunden wird die Liste der aktuell 50 beliebtesten Livestreams aktualisiert. Die Beliebtheit der Livestreams ändert sich nur leicht zwischen den Updates.

.....

.....

.....

.....

.....

.....

.....

- (b) (3 Punkte) Eine Wetter-App soll täglich weltweit Millionen von Wetterstationen nach Temperaturwerten sortieren, um Ausreißer oder Extremwerte effizient zu identifizieren. Die Temperaturdaten sind kontinuierlich verteilt, von $-50\text{ }^{\circ}\text{C}$ bis $+50\text{ }^{\circ}\text{C}$.

.....

.....

.....

.....

.....

.....

.....

- (c) (3 Punkte) Ein Compiler analysiert die Abhängigkeiten von Quellcodedateien und sortiert sie nach der Anzahl ihrer Abhängigkeiten, um die Kompilierung effizient zu planen. Die Dateiliste ist groß und vollständig unsortiert. Wenn mehrere Algorithmen gleich gut sind, wählen Sie den Algorithmus mit dem kleinsten Speicherbedarf.

.....

.....

.....

.....

.....

.....

.....

11. (11 Punkte) Das Partitionsproblem besteht darin, zu bestimmen, ob eine gegebene Menge von Ganzzahlen in zwei Teilmengen aufgeteilt werden kann, so dass die Summe der Elemente in beiden Teilmengen gleich ist.

Bspw. kann die Liste [2, 4, 2] in [4] und [2, 2] partitioniert werden, da die Summe der Elemente in beiden Listen 4 beträgt.

Die Brute-Force Lösung lautet:

```
def subsetSum(nums, n, total):
    if total == 0:
        return True

    if n < 0 or total < 0:
        return False

    include = subsetSum(nums, n - 1, total - nums[n])

    if include:
        return True

    exclude = subsetSum(nums, n - 1, total)

    return exclude

def partition(nums):
    total = sum(nums)

    return (total & 1) == 0 and subsetSum(nums, len(nums) - 1, total/2)

if __name__ == '__main__':
    nums = [2, 4, 2]

    if partition(nums):
        print('Set can be partitioned')
    else:
        print('Set cannot be partitioned')
```

Zeichnen Sie dazu den Rekursionsbaum der Methode **subsetSum** für das im Code aufgerufene Beispiel. Damit Sie nicht so viel schreiben müssen, würde ich Ihnen empfehlen in die Knoten nur die beiden Argumente **n** und **total** einzuzeichnen. Der Wurzelknoten des Rekursionsbaums sieht also so aus:



Rechenkästchen folgen auf der nächsten Seite.

